NPTEL Software Testing

Complete Exam Preparation Guide

Deep explanations | Worked examples | Memory tricks | 100 MCQs

Coverage	Input partitioning, graph testing, data flow, logic, mutation, OO, regression, non-functional, web and mobile testing
How to use	Read the summary boxes, solve the worked examples, then attempt the MCQs without checking the final answer page.
Source basis	Prepared from the supplied NPTEL course PDFs and exercise material.

Rapid Exam Map

Software testing can be organized around one question: **what must the tests cover?** Input space criteria cover representative inputs; graph criteria cover structure and paths; data-flow criteria cover definitions reaching uses; logic criteria cover Boolean decisions; mutation criteria cover seeded faults; system-level testing covers quality attributes and platform risks.

Area	Core question	Key terms
ISP	Which input categories and combinations?	Completeness, disjointness, ACoC, ECC, PWC, BCC
Graphs	Which nodes, edges and paths?	NC, EC, EPC, PPC, touring
Data flow	Which definitions reach which uses?	def, use, du-pair, def-clear, all-defs
Logic	Which clauses control a decision?	PC, CC, CoC, GACC, CACC, RACC
Mutation	Can tests distinguish a faulty variant?	kill, equivalent mutant, mutation score, RIP
System	Does quality hold beyond functionality?	load, stress, security, reliability, regression

Memory trick: I-G-D-L-M-S: Inputs, Graphs, Data, Logic, Mutation, System.

1. Foundations: Faults, Errors, Failures and RIPR

Testing is not the same as debugging. Testing executes software to expose failures or obtain evidence about quality. Debugging starts from a failure and searches for the responsible fault.

Fault, error state, and failure

A **fault** is a defect in code or design. Executing it may create an incorrect internal state called an **error**. If that error reaches an observable output, a **failure** occurs. A fault can execute without visible failure because the state may accidentally remain correct or the error may be overwritten.

```
Fault: loop uses i > 0 instead of i >= 0. Input [7] reaches the loop condition, skips index 0, returns -1, and produces a failure.
```

Memory trick: Code has the fault; state carries the error; user sees the failure.

RIPR model

A test reveals a fault only when it satisfies four conditions: **Reachability** of the fault, **Infection** of program state, **Propagation** of the error to output, and **Revelation** by the oracle. Weak oracles often fail at the last step.

```
R: execute faulty statement. I: faulty and correct states differ. P: difference survives. R: assertion checks the differing output.
```

Memory trick: RIPR = Reach, Infect, Propagate, Reveal.

Test oracle and automation

A test oracle decides pass or fail. Automation is excellent for repeatable execution and checking explicit assertions, but a weak oracle can automate the wrong check perfectly. Coverage measures execution obligations; it does not prove absence of faults.

Memory trick: Coverage asks what ran; the oracle asks whether the result was right.

2. Input Space Partitioning (ISP)

ISP divides the input domain into blocks whose members are expected to behave similarly. The model is useful only when its blocks are complete and disjoint.

Characteristics, blocks, completeness, disjointness

A **characteristic** describes an aspect of the input domain; its **blocks** are categories. Completeness means the union of blocks covers the entire domain. Disjointness means no value belongs to more than one block. Include valid, invalid, boundary, null, empty, and state-dependent inputs where relevant.

```
For age valid 18..60: B1 age<18, B2 18..60, B3 age>60. These blocks cover every integer and do not overlap.
```

Memory trick: Complete = nothing missing. Disjoint = no overlap.

Interface-based and functionality-based models

Interface-based modelling considers each parameter separately and is easier, but can miss interactions. Functionality-based modelling derives characteristics from requirements, preconditions, postconditions, and relationships among parameters. It is harder but often captures behavior more directly.

```
search(list,key): interface blocks include list null/empty/nonempty and key null/non-null. Functional blocks include key absent, present once, present multiple times.
```

Memory trick: Interface = individual parameters; Functionality = behavior.

ACoC and ECC

All Combinations Coverage uses every block combination. If characteristics have B1..Bn blocks, the count is the product. **Each Choice Coverage** uses every block of every characteristic at least once; its minimum is at least the largest block count, but actual feasibility may require more.

```
Blocks 2 x 3 x 2: ACoC = 12 tests. ECC may use 3 tests if each block appears at least once.
```

Memory trick: ACoC multiplies; ECC touches each choice.

Pair-wise and T-wise coverage

Pair-wise requires every pair of blocks from every pair of characteristics to occur in some test. T-wise generalizes this to every t-way combination. When t equals the number of characteristics, T-wise becomes ACoC. Pair-wise is cheaper but may miss faults requiring three-way or higher interaction.

```
Browser {Chrome,Edge}, OS {Win,Linux}, role {admin,user}: select tests so every Browser-OS,
Browser-role and OS-role pair appears.
```

Memory trick: Pair-wise covers handshakes between two characteristics.

BCC, MBCC and constraints

Base Choice Coverage selects one important block per characteristic to form a base test, then changes one characteristic at a time. Test count is 1 + sum(Bi-1). MBCC allows multiple base choices. Infeasible combinations must be excluded or the base altered.

```
Payment base = valid card, normal amount, online. Add tests varying only card to expired, amount
to zero/high, and network to offline.
```

Memory trick: BCC = one stable base, one change at a time.

3. Structural Graph Coverage

A graph has nodes, edges, initial nodes and final nodes. A test path starts at an initial node and ends at a final node. Coverage criteria convert graph structure into test requirements.

Paths and test paths

A path is a sequence of adjacent nodes. A simple path does not repeat a node except that first and last may match. A prime path is a simple path that is not a proper subpath of another simple path. A complete path begins at an initial node and ends at a final node.

```
Graph edges 1-2, 2-3, 2-4, 3-2. [1,2,4] is a test path. [2,3,2] is a round trip but not a test
path if 2 is not initial/final.
```

Memory trick: Prime = maximal simple path.

Node, edge and edge-pair coverage

Node Coverage tours every reachable node. Edge Coverage tours every reachable edge and subsumes NC in ordinary connected control-flow graphs. Edge-Pair Coverage tours every reachable length-2 subpath. Stronger coverage creates more obligations but still does not guarantee fault detection.

```
For edges (1,2),(2,3),(3,2),(2,4), path [1,2,3,2,4] covers all edges but misses edge-pair [3,2,3].
```

Memory trick: Node = places; Edge = moves; Edge-pair = two consecutive moves.

Prime Path Coverage

PPC requires every feasible prime path to be toured. It captures maximal acyclic paths and simple cycles and is often substantially stronger than EC or EPC. Loops can make the number of complete paths infinite, but the set of prime paths is finite for a finite graph.

Memory trick: PPC remembers the longest simple routes.

Direct touring, sidetrips and detours

A test path tours a required path directly when the required node sequence occurs contiguously. A sidetrip leaves and returns to the same node while preserving the required edge order; a detour may replace an edge with another path. Best-effort touring may accept weaker forms when direct touring is infeasible.

Memory trick: Direct is exact; sidetrip returns; detour replaces an edge.

CFG construction

Decisions create branches; loops create a back edge and an exit edge; break jumps to loop exit; continue jumps to the next iteration check; return jumps to method exit. Short-circuit Boolean expressions may need separate nodes when modelling clause evaluation.

```
if (x>0) y++; else y--; return y; creates a decision node, two branch nodes, a merge, and exit.
```

Memory trick: One node per meaningful control decision or straight-line block.

4. Data-Flow Testing

Data-flow testing follows the life of a variable from definition to use. It is especially good at finding initialization, stale-value, overwritten-value and wrong-use faults.

Definitions and uses

A definition assigns a value. A computational use (c-use) reads it in an expression or output. A predicate use (p-use) reads it in a decision. A definition at node d reaches use u if there is a path from d to u with no intervening redefinition.

```
x=5 is a def. y=x+1 is a c-use of x. if(x>0) is a p-use.
```

Memory trick: Define, then use; no redefinition in between.

DU-pairs and def-clear paths

A du-pair is a reachable pair (d,u) for the same variable. A du-path is a simple, def-clear path from d to u. If the variable is redefined in the middle, the old definition no longer reaches that use.

```
1:x=0; 2:if(a); 3:x=1; 4:print(x). Path 1-2-4 is def-clear for (1,4); path 1-2-3-4 is not.
```

Memory trick: DU = where value is born and where it is consumed.

All-defs, all-uses, all-du-paths

All-defs covers at least one use reachable from every definition. All-uses covers every reachable use for every definition. All-du-paths covers every simple def-clear path for every du-pair. The usual strength order is all-du-paths, then all-uses, then all-defs.

Memory trick: One use per def; every use per def; every clear path per pair.

Anomalies and interprocedural flow

Typical suspicious patterns include define-define without use, use before definition, and define-kill without use. Across methods, coupling data flow connects actual parameters and object state at call sites with formal parameters, returns, and effects in callees.

Memory trick: d-u is useful; u before d and d-d deserve attention.

5. Specification-Based Graph Testing

Graphs can model not only code but also requirements, states, events, use cases and sequencing constraints.

Finite-state machines

An FSM has states and labelled transitions. Tests can cover states, transitions, transition pairs, or longer transition sequences. A test must include a prefix that reaches the required pre-state before applying the triggering event.

```
Turnstile states Locked/Unlocked. coin: Locked->Unlocked. push: Unlocked->Locked. Test [coin,push]
covers both transitions from the initial Locked state.
```

Memory trick: State says where you are; event says how you move.

Sequencing constraints

Specifications often state that A must occur before B, B must not occur after C, or an action is allowed only in a state. Convert these into graph constraints and create both valid and invalid sequences. Static checks inspect structural rules while dynamic checks execute sequences.

Memory trick: PRE: reach the state, EVENT: trigger, ORACLE: verify next state.

Use-case and call graphs

Use-case scenarios model user-visible flows including alternate and exceptional paths. Call graphs model possible method-to-method calls and support node, edge and prime-path reasoning at the integration level.

Memory trick: Scenario graphs test stories; call graphs test collaborations.

6. Logic Coverage

Logic criteria test Boolean predicates and their clauses. The key idea is whether changing one clause can change the predicate.

Predicates and clauses

A predicate evaluates to true or false. A clause contains no logical operator. In p=(x>y) OR C OR f(z), the three clauses are x>y, C, and f(z). Predicate Coverage makes p true and false. Clause Coverage makes every clause true and false, but may leave p with only one outcome.

Memory trick: PC flips the decision; CC flips each ingredient.

Combinatorial Coverage

For n clauses, CoC requires all 2^n truth assignments. It is strongest among the basic table-based criteria but grows exponentially and may include infeasible combinations.

```
Three independent clauses need 8 truth assignments: TTT through FFF.
```

Memory trick: CoC = complete truth table.

Major clause and determination

Choose one clause as the major clause. It determines p when flipping only that clause, under suitable minor-clause values, flips p. Each clause takes a turn as major. For a OR b, a determines p only when b is false.

```
For p=a AND (b OR c), a determines p when (b OR c)=true. b determines p when a=true and c=false.
```

Memory trick: Determine means the major clause has control.

GACC, CACC and RACC

GACC requires the major clause true and false while determining p; minor values may differ. CACC additionally requires p to have opposite outcomes. RACC requires minor clauses to be identical in the paired tests, so the major clause is the only change. CACC subsumes PC; GACC need not.

```
p=a OR b, major a: paired tests (T,F) and (F,F) satisfy RACC, CACC and GACC.
```

Memory trick: G = minors may vary; C = predicate correlates; R = minors remain rigid.

Inactive clause coverage

ICC checks situations where changing the major clause does not affect p. GICC allows minor values to vary; RICC fixes them within each pair. It is valuable when a condition should be irrelevant in a mode.

Memory trick: Active asks influence; inactive asks deliberate non-influence.

Feasibility and MC/DC

Constraints may make truth-table rows impossible. RACC can be infeasible even when CACC is feasible. MC/DC is commonly treated as active clause coverage: each condition must independently affect the decision outcome.

Memory trick: First establish reachability and feasibility, then select truth assignments.

7. Mutation Testing

Mutation testing creates small syntactic variants called mutants and checks whether the test suite distinguishes them from the original program.

Killing mutants and mutation score

A mutant is killed when some test yields observably different behavior from the original. A live mutant may indicate a weak test, an equivalent mutant, or an unreachable mutant. Mutation score is killed non-equivalent mutants divided by total non-equivalent mutants.

```
Original: a+b. Mutant: a-b. Test a=5,b=2 gives 7 vs 3, so it kills the mutant.
```

Memory trick: Kill = make mutant visibly disagree.

Mutation operators

Common operators replace arithmetic, relational, logical or conditional operators; alter constants; delete statements; negate conditions; or replace variables. OO and integration mutation can change method calls, inheritance behavior, polymorphic binding, parameters, or returned values.

Memory trick: A-R-L-C-V: Arithmetic, Relational, Logical, Constant, Variable.

Strong and weak mutation

Strong mutation requires the infected state to propagate to observable output. Weak mutation checks state immediately after the mutated expression, so propagation is not required. Both still need reachability and infection.

Memory trick: Weak stops after infection; strong waits for output.

Equivalent mutants

An equivalent mutant behaves identically for all inputs and cannot be killed. Detecting equivalence is generally difficult, so mutation analysis often requires manual reasoning or approximations. Avoid counting confirmed equivalent mutants in the denominator.

Memory trick: Live does not always mean weak test; it may mean equivalent.

Integration and specification mutation

Mutation can target interfaces and grammars, not just source expressions. Grammar mutation creates valid or invalid input forms; integration mutation changes calls or parameter interactions. The goal remains the same: design tests sensitive to realistic fault patterns.

Memory trick: Mutate the contract boundary, not only the method body.

8. Object-Oriented Testing

OO systems add faults involving inheritance, overriding, dynamic binding, object state and collaboration.

Inheritance and overriding faults

A subclass may violate assumptions of its superclass, misuse inherited state, hide fields, or override a method inconsistently. Tests should run inherited methods on subclass objects and compare behavior across the hierarchy.

Memory trick: Test base behavior, overridden behavior, and cross-type behavior.

Polymorphism and dynamic binding

The method executed depends on the runtime type, not merely the declared reference type. Cover relevant receiver-type and argument-type combinations, including overridden methods and super calls.

```
Shape s=new Circle(); s.area() must execute Circle.area(), not Shape.area().
```

Memory trick: Declared type points; runtime type chooses.

Contracts and equality

For equality-like methods, check reflexivity, symmetry, transitivity, consistency and null handling. If equal objects must work in hash structures, equal objects must have equal hash codes. Inheritance can make symmetry and substitutability difficult.

Memory trick: EQUAL: reflexive, symmetric, transitive, consistent, null-safe.

Coupling coverage and yo-yo graphs

Integration tests should cover call sites and coupling du-pairs between callers and callees. Deep inheritance with repeated overriding can produce a yo-yo effect in which control moves up and down the hierarchy, making behavior harder to reason about.

Memory trick: Follow values across calls and dispatch across classes.

9. Regression Testing

Regression testing gives confidence that changes did not break previously working behavior.

Retest-all, selection, minimization, prioritization

Retest-all reruns the complete suite and is safest but costly. Regression test selection chooses tests affected by changes. Minimization removes tests while preserving selected coverage and may lose fault diversity. Prioritization orders tests to obtain earlier feedback without necessarily discarding tests.

Memory trick: Select = choose affected; minimize = remove redundancy; prioritize = reorder.

Safe selection and change impact

A safe technique aims not to omit tests that could reveal faults caused by the change, according to its model and assumptions. Impact analysis uses changed code, dependencies, call graphs, data flow and configuration. Dynamic-language features, reflection, environment changes and hidden dependencies can reduce precision.

Memory trick: Changed code plus dependencies define the regression cone.

Automation and maintenance

Regression automation needs stable setup, deterministic data, isolated tests, meaningful oracles and failure diagnosis. Flaky tests damage trust. Update tests when intended behavior changes; do not blindly rewrite assertions just to obtain green results.

Memory trick: A reliable small suite is better than a large flaky suite.

10. Non-Functional Testing

Non-functional testing evaluates quality attributes and operational behavior rather than only business outputs.

Performance family

Load testing checks expected workload; stress testing pushes beyond capacity; spike testing applies sudden changes; soak testing runs sustained load to expose leaks and degradation; scalability testing measures behavior when resources or workload grow.

```
API target 500 users: load at 500, stress beyond 500 until failure, spike 50->1000 rapidly, soak at 500 for long duration.
```

Memory trick: Load=normal, Stress=beyond, Spike=sudden, Soak=sustained.

Reliability and recovery

Reliability concerns failure-free operation over a period or workload. Recovery testing checks restart, failover, rollback, checkpoint and data integrity after faults. Availability depends on both failure behavior and restoration.

Memory trick: Reliability avoids failure; recovery limits damage and downtime.

Security testing

Cover authentication, authorization, input validation, session management, confidentiality, integrity, logging and abuse cases. Verify negative cases such as horizontal privilege escalation and direct access to protected resources.

Memory trick: AuthN = who are you; AuthZ = what may you do.

Interoperability, usability and documentation

Interoperability tests data formats, protocols, versions and error handling across components. Usability checks effectiveness and accessibility for intended users. Documentation testing verifies correctness, completeness, examples, installation steps and consistency with the product.

Memory trick: The product includes interfaces, user experience and instructions.

11. Web Application Testing

Web systems combine browser behavior, network protocols, server logic, state, databases and security boundaries.

Static and dynamic content

Static pages mainly need link, content, layout and compatibility checks. Dynamic pages need request/response, session, database, authorization and concurrency testing. Client-side validation improves usability but cannot replace server-side validation.

Memory trick: Anything trusted by the server must be validated by the server.

Client, server and bypass testing

Test JavaScript and DOM behavior in the browser, server behavior through normal UI flows, and bypass paths by sending crafted requests directly. Parameters, cookies, headers and hidden fields are attacker-controlled inputs.

Memory trick: UI tests the path; API tests the contract; bypass tests the boundary.

Compatibility and resilience

Cover supported browsers, viewport sizes, cookies, caching, navigation, refresh, back button, duplicate submission, timeout, network loss and partial server failure. Verify idempotency where repeated requests are possible.

Memory trick: Web state lives across browser, network, server and database.

12. Mobile Application Testing

Mobile testing adds device fragmentation, lifecycle events, sensors, permissions, connectivity and constrained resources.

Device and OS fragmentation

Test supported OS versions, screen sizes, densities, orientations, locales and hardware capabilities. Use emulators for broad fast coverage and real devices for hardware, performance and platform behavior.

Memory trick: Emulator for breadth; real device for truth.

Lifecycle and interruptions

Cover install, first launch, background/foreground, rotation, process death, upgrade, uninstall/reinstall, calls, notifications, alarms and low-memory conditions. App state should be preserved or safely reconstructed.

Memory trick: Pause, kill, rotate, resume: state must survive correctly.

Network, permissions and resources

Test offline mode, weak network, switching Wi-Fi/cellular, latency, retries, duplicate requests and conflict handling. Verify permission denial, later grant/revoke, battery, storage, CPU and memory behavior.

Memory trick: Assume the network disappears and permission is refused.

Mobile security and usability

Protect local storage, tokens, logs, screenshots and inter-app communication. Check touch target size, gestures, keyboards, accessibility, responsive layout and clear recovery messages.

Memory trick: Secure data at rest, in transit and on screen.

13. Final Formula and Comparison Sheet

Concept	Exam-ready statement
ACoC count	Product of the number of blocks in every characteristic.
ECC minimum	At least the maximum block count, subject to feasibility.
BCC count	1 + sum(Bi - 1).
CoC logic count	2^n truth assignments for n independent clauses.
PC vs CC	PC flips predicate; CC flips each clause. Neither generally subsumes the other.
CACC	Major clause determines predicate and predicate outcomes differ.
RACC	CACC-style pair with identical minor-clause values.
All-defs	At least one reachable use for every definition.
All-uses	Every reachable use for every definition.
All-du-paths	Every simple def-clear path for every du-pair.
Mutation score	Killed non-equivalent mutants / total non-equivalent mutants.
Regression	Selection chooses, minimization removes, prioritization reorders.

100 Practice MCQs

Choose the best answer. Do not check the key until you finish all questions. The complete answer key is placed on the final page.

1. A defect in source code is called a:
A. failure
B. fault
C. oracle
D. test path

2. An incorrect internal program state is an:
A. error
B. edge
C. input block
D. mutant score

3. In RIPR, the final R means:
A. retest
B. revelation
C. restriction
D. recovery

4. A test executes a fault but state remains correct. Which RIPR condition failed?
A. Reachability
B. Infection
C. Propagation
D. Revelation

5. A weak assertion most directly harms:
A. reachability
B. infection
C. revelation
D. partitioning

6. Testing differs from debugging because debugging primarily:
A. generates partitions
B. locates and repairs faults after failure
C. measures load
D. covers edges

7. Coverage evidence alone proves:
A. absence of faults
B. all outputs correct
C. only that specified obligations executed
D. all mutants equivalent

8. The user-visible incorrect behavior is a:
A. definition
B. failure
C. clause
D. base choice

9. A test oracle determines:
A. which code compiles
B. pass or fail
C. graph nodes
D. runtime type

10. A propagated error not checked by the oracle fails at:
A. Reachability
B. Infection
C. Propagation
D. Revelation

11. For valid partitions, blocks must be:
A. random and equal-sized
B. complete and disjoint
C. numeric and ordered
D. all valid

12. Interface-based ISP mainly considers:
A. each parameter separately
B. only requirements
C. only invalid inputs
D. method calls

13. Functionality-based ISP mainly derives characteristics from:
A. file names
B. requirements and behavior
C. compiler output
D. CPU usage

14. For block counts 2,3,4, ACoC requires:
A. 4
B. 9
C. 12
D. 24

15. ECC requires:
A. every combination
B. every block at least once
C. every pair of paths
D. only base tests

16. Pair-wise coverage focuses on:
A. all 2-way block interactions
B. two test cases only
C. two values per input
D. all paths of length two

17. T-wise becomes ACoC when t equals:
A. 2
B. largest block count
C. number of characteristics
D. number of tests

18. BCC varies:
A. all characteristics simultaneously
B. one characteristic from a base at a time
C. only invalid blocks
D. only outputs

19. With blocks 3,2,4, BCC count is:
A. 7
B. 8
C. 9
D. 24

20. An impossible block combination is:
A. complete
B. infeasible
C. prime
D. equivalent

21. A test path must start at an initial node and:
A. repeat every node
B. end at a final node
C. contain a cycle
D. be prime

22. A prime path is:
A. any complete path
B. a maximal simple path
C. a path of one edge
D. an infeasible path

23. Which is stronger in the usual graph hierarchy?
A. Node Coverage over Edge Coverage
B. Edge Coverage over Node Coverage
C. Node and Edge always identical
D. Neither has requirements

24. Edge-Pair Coverage requires paths of length:
A. 0 edges

B. 1 edge
C. 2 edges
D. all edges

25. A direct tour means the required path appears:
A. in reverse
B. contiguously
C. only with a detour
D. in another method

Practice MCQs 26-50

26. A sidetrip typically:
A. leaves and returns while preserving required order
B. removes a node
C. changes the final node
D. makes a path simple

27. A continue statement in a loop transfers control to:
A. method exit
B. next loop condition/update
C. previous method
D. exception handler only

28. Complete Path Coverage may be impractical because loops can produce:
A. no paths
B. infinitely many test paths
C. one node
D. only prime paths

29. PPC stands for:
A. Predicate Pair Coverage
B. Prime Path Coverage
C. Program Partition Criterion
D. Pair Path Count

30. Edge Coverage can still miss:
A. all nodes
B. faults requiring particular edge sequences
C. compilation errors
D. every branch

31. A statement x=7 is a:
A. use
B. definition
C. predicate
D. final node

32. A use in y=x+1 is a:
A. p-use
B. c-use
C. definition
D. kill

33. A use in if(x>0) is a:
A. p-use
B. c-use only
C. definition
D. mutant

34. A def-clear path contains:
A. no use
B. no redefinition between def and use
C. no branches
D. no nodes

35. A du-pair connects:
A. two definitions
B. a definition and reachable use
C. two predicates
D. two mutants

36. The weakest of these is usually:
A. all-du-paths
B. all-uses
C. all-defs
D. combinatorial coverage

37. All-uses requires:
A. one use per program
B. every reachable use for each definition
C. every path in graph

D. all inputs

38. A redefinition between d and u:
A. preserves old def
B. kills the old reaching definition
C. creates PC
D. makes a prime path

39. Use before definition is a potential:
A. data-flow anomaly
B. base choice
C. load test
D. equivalent mutant

40. Coupling du-pairs are most relevant to:
A. interprocedural testing
B. font testing
C. ACoC
D. spike testing

41. An FSM transition is usually labelled by:
A. an event or condition
B. a block count
C. a mutation score
D. a file extension

42. Before testing a transition, a test may need a prefix to:
A. compile source
B. reach the pre-state
C. kill a mutant
D. form a clause

43. Transition coverage on an FSM is analogous to:
A. edge coverage
B. clause coverage
C. all-defs
D. ECC

44. Use-case scenarios are especially useful for:
A. user-visible flows
B. CPU arithmetic only
C. equivalent mutants
D. hash codes only

45. A call graph node commonly represents a:
A. method/function
B. input block
C. truth value
D. browser cookie

46. A predicate is an expression that returns:
A. a file
B. Boolean value
C. graph only
D. mutant

47. A clause contains:
A. no logical operator
B. at least two ANDs
C. a complete path
D. a definition and use

48. Predicate Coverage requires each predicate:
A. true only
B. false only
C. true and false
D. all 2^n combinations

49. Clause Coverage guarantees Predicate Coverage:
A. always
B. never under any circumstance
C. not in general
D. only for 10 clauses

50. For n independent clauses, CoC needs:
A. n
B. n+1
C. 2n
D. 2^n

Practice MCQs 51-75

51. For p=a OR b, a determines p when b is:
A. true
B. false
C. undefined
D. both true and false simultaneously

52. The clause currently being tested for influence is the:
A. minor clause
B. major clause
C. base clause
D. final clause

53. GACC permits minor-clause values to:
A. never change
B. differ between paired tests
C. be omitted from predicate
D. all be true only

54. CACC additionally requires the predicate outcomes to:
A. remain equal
B. differ
C. be infeasible
D. be ignored

55. RACC requires minor clauses in the pair to be:
A. identical
B. all true
C. all false
D. unreachable

56. Which criterion checks non-influence?
A. ACC
B. ICC
C. ACoC
D. EC

57. MC/DC is closely related to:
A. active clause coverage
B. load testing
C. all-defs
D. web caching

58. Constraints can make RACC:
A. automatic
B. infeasible
C. the same as ECC
D. a graph node

59. CACC subsumes:
A. Predicate Coverage
B. all-du-paths
C. ACoC
D. stress testing

60. GACC necessarily subsumes PC:
A. true
B. false
C. only for mobile apps
D. only for no clauses

61. A mutant is killed when:
A. it compiles
B. a test distinguishes it from original
C. it has no operator
D. it is equivalent

62. Mutation score excludes confirmed:
A. killed mutants
B. equivalent mutants
C. tests

D. operators

63. Replacing + with - is an:
A. arithmetic operator replacement
B. edge pair
C. base choice
D. load model

64. A live mutant may indicate:
A. weak tests or equivalence
B. perfect correctness
C. no source code
D. only a syntax error

65. Strong mutation requires error difference to:
A. exist immediately only
B. propagate to observable output
C. avoid reachability
D. be a partition

66. Weak mutation checks difference:
A. near mutated location
B. only at final output
C. in requirements only
D. after deployment

67. Equivalent mutants are difficult because they:
A. always crash
B. behave identically on all inputs
C. cannot compile
D. always change output

68. Statement deletion is a type of:
A. mutation operator
B. graph node
C. state machine
D. oracle

69. Integration mutation primarily targets:
A. calls and interactions
B. screen size only
C. fonts
D. page numbering

70. The fundamental aim of mutation is to assess:
A. test sensitivity to faults
B. developer rank
C. network bandwidth
D. document quality

71. Dynamic binding selects a method using the:
A. runtime receiver type
B. file name
C. declared variable name only
D. test count

72. An overriding fault occurs between:
A. superclass and subclass methods
B. two input blocks
C. two graph edges
D. two browsers

73. equals symmetry means:
A. x.equals(y) equals y.equals(x)
B. x equals null
C. all objects equal
D. hash codes differ

74. If two objects are equal, their hash codes must be:
A. different
B. equal
C. negative
D. prime

75. A yo-yo graph is associated with:
A. deep inheritance and overriding
B. pair-wise ISP
C. stress testing
D. web links

Practice MCQs 76-100

76. Polymorphism tests should vary:
A. runtime receiver types
B. only filenames
C. only test order
D. only screen color

77. Coupling coverage examines:
A. interactions across calls
B. only local constants
C. only UI text
D. page layout

78. Substitutability problems arise when a subtype:
A. violates expected supertype behavior
B. has a constructor
C. uses a variable
D. is compiled

79. Composition is sometimes preferred to inheritance when:
A. equality/contracts become inconsistent
B. ACoC is small
C. a graph has no edge
D. load is normal

80. Retest-all means:
A. rerun the whole test suite
B. run only failed tests
C. drop redundant tests
D. sort tests alphabetically

81. Regression test selection:
A. chooses affected tests
B. always runs all tests
C. changes requirements
D. measures load

82. Test minimization may risk losing:
A. fault-detection diversity
B. compilation
C. all source files
D. runtime types

83. Prioritization mainly:
A. reorders tests for earlier feedback
B. deletes all slow tests
C. changes outputs
D. creates mutants

84. A safe selection technique aims not to omit:
A. tests that could reveal change-related faults
B. all equivalent mutants
C. all users
D. all documents

85. A flaky test is one that:
A. passes and fails nondeterministically
B. always fails correctly
C. has no assertion
D. uses a base choice

86. Change impact analysis may use:
A. dependencies and call graphs
B. only font size
C. only page count
D. only clause truth tables

87. When intended behavior changes, regression tests should:
A. be updated deliberately with requirement evidence
B. be ignored
C. all be deleted

D. always retain old oracle

88. Load testing evaluates:
A. expected workload
B. only sudden spikes
C. only recovery
D. only invalid inputs

89. Stress testing pushes the system:
A. beyond expected capacity
B. only at zero load
C. through truth tables
D. through inheritance

90. Spike testing applies:
A. sudden workload changes
B. long stable workload only
C. mutation operators
D. input partitions only

91. Soak testing is designed for:
A. sustained execution and degradation
B. one request
C. source compilation
D. clause pairing

92. Authentication answers:
A. who are you
B. what may you do
C. how fast is it
D. which path is prime

93. Authorization answers:
A. what may you do
B. who are you
C. how many blocks
D. which mutant is equivalent

94. Recovery testing checks:
A. restart/failover and data integrity
B. only UI color
C. only pair-wise inputs
D. only node coverage

95. Interoperability testing focuses on:
A. components, protocols and formats working together
B. only local variables
C. only classes
D. only load

96. Client-side validation should be backed by:
A. server-side validation
B. no validation
C. only CSS
D. only caching

97. Hidden fields and cookies should be considered:
A. untrusted inputs
B. trusted because browser-generated
C. graph nodes only
D. equivalent mutants

98. Bypass testing sends requests:
A. without relying only on normal UI flow
B. only through screenshots
C. only while offline
D. only after mutation

99. A mobile emulator is best used for:
A. broad fast configuration coverage
B. all hardware truth
C. battery chemistry
D. final real-device certainty

100. Real mobile devices are important for:
A. hardware and platform-realistic behavior
B. truth tables only
C. prime paths only
D. ACoC formulas

Answer Key - Final Page

Use this key only after completing all 100 questions. For missed questions, return to the matching topic and explain why each distractor is wrong.

1 B	**2** A	**3** B	**4** B	**5** C	**6** B	**7** C	**8** B	**9** B	**10** D
11 B	**12** A	**13** B	**14** D	**15** B	**16** A	**17** C	**18** B	**19** A	**20** B
21 B	**22** B	**23** B	**24** C	**25** B	**26** A	**27** B	**28** B	**29** B	**30** B
31 B	**32** B	**33** A	**34** B	**35** B	**36** C	**37** B	**38** B	**39** A	**40** A
41 A	**42** B	**43** A	**44** A	**45** A	**46** B	**47** A	**48** C	**49** C	**50** D
51 B	**52** B	**53** B	**54** B	**55** A	**56** B	**57** A	**58** B	**59** A	**60** B
61 B	**62** B	**63** A	**64** A	**65** B	**66** A	**67** B	**68** A	**69** A	**70** A
71 A	**72** A	**73** A	**74** B	**75** A	**76** A	**77** A	**78** A	**79** A	**80** A
81 A	**82** A	**83** A	**84** A	**85** A	**86** A	**87** A	**88** A	**89** A	**90** A
91 A	**92** A	**93** A	**94** A	**95** A	**96** A	**97** A	**98** A	**99** A	**100** A

Last-minute focus: memorize completeness/disjointness; ACoC, ECC, PWC and BCC; NC/EC/EPC/PPC; def-use hierarchy; PC/CC/CoC/GACC/CACC/RACC; mutation score and equivalent mutants; and the load-stress-spike-soak distinction.